UNIVERSIDAD EIA
Ser, Saber y Servir

Object-Oriented Programming — Applied Exercise

The Hospital de los Costeños con Narcolepsia

Course	Object-Oriented Programming — Universidad EIA
Instructor	Sebastián Zapata Ramírez
Language	Go (Golang) 1.21 or later — no other language is accepted
Groups	1 student
Due date	19 / 10 / 2026, 23:59 (repository link)
Oral defense	20 / 10 / 2026 — individual, mandatory, see Section 8
Weight	15 % of the final course grade

1. Introduction: The Origin of the Name

Legend has it that the warm coastal breeze, combined with a heavy sancocho de pescado, creates an unstoppable lethargic effect. But what happens when you combine the loud, vibrant and famously energetic spirit of a Colombian costeño with clinical narcolepsy? Absolute, unpredictable chaos.

One minute they are dancing champeta at full volume or arguing passionately about baseball, and the very next second they are sound asleep standing up against a palm tree. The "Hospital de los Costeños con Narcolepsia" was founded to address this very specific and highly tragic paradox: treating people whose souls want to party at the carnival until 6:00 AM, but whose nervous systems completely shut down at 2:15 PM in the middle of a sentence. Your mission, as the software engineer of this facility, is to bring digital order to this very sleepy carnival.



The two natural states of the patient: eating sancocho, and arriving at the hospital.

1.1 What you are building

You must develop the central management system of the hospital. The institution treats patients from the coast who suffer from narcolepsy and therefore fall asleep suddenly anywhere in the facilities.

The system must track where each patient is, which doctor is attending them, and record every sudden sleep episode so that an available bed can be assigned quickly, before the patient blocks a hallway.

The deliverable is a program with a `main` function that runs the demonstration scenario of Section 6 and prints the results of the four queries of Section 5. A graphical interface is optional and gives bonus points (Section 7).

2. Technical Rules

- The whole exercise must be written strictly in **Go (Golang)**, version 1.21 or later, using only the standard library. Third-party modules are allowed only for the optional graphical interface.
- Go is not a classical object-oriented language: it has no classes and no traditional inheritance. You must express the model with **structs, methods with receivers, struct composition (embedding) and interfaces**, which is how Go achieves encapsulation and polymorphism.
- Encapsulation is enforced by the package system: fields that must not be touched from outside are written in **lowercase** (unexported) and are read or modified through methods. A struct with every field exported and no methods is not an object model and will be graded as such.
- Methods that can fail must return an `error` as their last value. Do not use `panic` for situations the hospital can foresee, such as "there is no free bed".
- The code must be formatted with `gofmt` and must pass `go vet` with no warnings.
- Suggested layout: a package `hospital` with the model, and a package `main` that only builds the scenario and prints. Business logic inside `main.go` is not accepted.

```
costenos-narcolepsia/
  go.mod
  main.go            // scenario + printing only
  hospital/
    hospital.go
    patient.go
    doctor.go
    room.go
    episode.go
    hospital_test.go
```

3. Entity Model

Implement at least these five structs. The names may be in English or Spanish, but they must be consistent across the whole project. Fields and methods listed below are the minimum; you may add more.

Struct	Minimum fields	Minimum methods

Hospital	`name`
	`doctors, patients, rooms`
	`history (all episodes)`
		`AdmitPatient(p) error`
		`HireDoctor(d) error`
		`AssignRoom(p) (*Room, error)`
		`RegisterEpisode(...) error`

Patient	`id, name, age`	`SufferSleepAttack(location)`

Struct	Minimum fields	Minimum methods
	narcolepsyLevel (Mild / Moderate / Severe) state (Awake / AsleepInHallway / AsleepInBed) currentLocation assignedDoctor	WakeUp() error State() PatientState
Doctor	id, name specialty patients under care episodes attended	DiagnosePatient(p) error AttendSleepEmergency(p) error MyEpisodes() []EpisodeRecord
Room	number, capacity state (Available / Occupied) occupants	IsAvailable() bool Occupy(p) error Release(p) error
EpisodeRecord	id date and time (time.Time) patient, attending doctor location where the patient fell asleep assigned room (may be empty)	Summary() string

3.1 States must be typed constants, not loose strings

Comparing bare strings such as "Asleep_In_Hallway" invites typos that the compiler cannot catch. Declare named types with constants:

```
type PatientState int

const (
    Awake PatientState = iota
    AsleepInHallway
    AsleepInBed
)

func (s PatientState) String() string { ... }   // for readable printing
```

Do the same for the narcolepsy level (Mild, Moderate, Severe) and for the room state (Available, Occupied).

4. Composition and Interfaces (this is the part that is really graded)

The exercise is about object orientation, so the model must show both mechanisms Go offers instead of inheritance. Two concrete requirements:

4.1 Composition (embedding)

Patients and doctors share identity data. Extract it into a struct `Person` (id, name, age) and **embed** it into `Patient` and `Doctor` — do not copy the same three fields twice.

```
type Person struct {
    id   string
    name string
```

```
    age  int
}

type Patient struct {
    Person                  // embedded: Patient reuses the Person methods
    level NarcolepsyLevel
    state PatientState
}
```

4.2 Interface (polymorphism)

Declare at least one interface that is genuinely used, that is, a place in the code where you hold values of different concrete types in the same variable or slice. The minimum required is:

```
type Attender interface {
    ID() string
    Name() string
    Attend(p *Patient, location string) (EpisodeRecord, error)
}
```

Implement it with `Doctor` and with a second type of your own — for example `Orderly` (camillero), who can move a sleeping patient to a bed but cannot diagnose. The hospital must store its staff in a `[]Attender` and dispatch whoever is available without knowing which concrete type it is. **An interface declared but never used polymorphically scores zero in this item.**

5. Required Queries

The entities must communicate with each other to answer these four queries. Each one must be a method that returns data — not a method that prints. Printing happens in main.

5.1 Patients asleep in a hallway

The hospital iterates over all its patients and returns those whose state is `AsleepInHallway`, so that an orderly can be dispatched.

```
func (h *Hospital) PatientsInHallway() []*Patient
```

5.2 History by doctor

A doctor returns every episode record in which they intervened. The doctor must not receive the hospital as a parameter: the relationship has to be modelled inside the entity.

```
func (d *Doctor) MyEpisodes() []EpisodeRecord
```

5.3 Bed availability

When a patient suffers a sleep attack, the system searches the rooms and assigns the first available one, changing its state to `Occupied` and the patient state to `AsleepInBed`. If no room is free, it must return an error and the patient stays in `AsleepInHallway` — the system must not crash and must not invent a room.

```
func (h *Hospital) AssignRoom(p *Patient) (*Room, error)
```

5.4 Severity report

Cross-reference two sources: the patients whose level is `Severe` and the number of episodes each of them had during the day. Return the pairing, not a printed table.

```
func (h *Hospital) SevereReport() map[*Patient]int
```

6. Demonstration Scenario (mandatory)

Your `main` must build and run exactly this scenario, in this order, so the evaluator can compare results across groups:

- Create the hospital, hire **2 doctors** and **1 orderly**, register **3 rooms with capacity 1** and admit **5 patients** (at least two of them with level Severe).
- Four patients suffer a sleep attack in different locations. Three of them get a room; the fourth one must fail to get one, stay in the hallway, and the error must be printed as a readable message.
- One of the patients wakes up, releases the room, and the patient left in the hallway is then assigned to that room.
- Print the result of the four queries of Section 5.

The exact wording of the output is up to you, as long as the four queries are clearly identifiable. An illustrative example:

```
== Patients asleep in hallway ==
  P-004  Wilfrido Berrio       (hallway 2, second floor)

== Episodes attended by Dr. Karen Ospina ==
  2026-05-14 14:15  P-001  cafeteria         -> room 101
  2026-05-14 15:02  P-003  radiology queue   -> room 103

== Assigning room to P-004 ==
  no room available: patient stays in the hallway

== Severity report (Severe patients) ==
  P-002  Yeimy Padilla     3 episodes today
  P-004  Wilfrido Berrio   2 episodes today
```

Include at least **three tests** in `hospital_test.go`: one for room assignment when there is a free room, one for the failure when there is none, and one for any of the queries.

7. Bonus Points (optional)

Graphical interface (+10 %). Couple the Go logic with a graphical interface using Fyne, Wails, or a lightweight REST API consumed by an HTML/JS frontend. What is rewarded is the **clean separation between logic and view**: the `hospital` package must not change at all when the interface is added, and must contain no code related to the display.

Concurrent simulation (+5 %). Launch each patient as a goroutine that falls asleep at random intervals, and protect the hospital state with a `sync.Mutex`. It must run clean under `go run -race .`; a version with data races does not earn the bonus.

8. Use of AI and Large Language Models

Using LLMs (Claude, ChatGPT, Copilot, or any other) is fully allowed for every part of this exercise: design, code, tests, documentation and the optional interface. There is no restriction and no penalty for using them.

There is exactly one condition: **you must be able to defend every line you deliver.** What is graded is not who typed the code, but whether you understand it, can justify it and can modify it. Code that no member of the group can explain is worth zero, no matter how well it runs.

8.1 The AI_USAGE.md file (mandatory)

Include in the repository a short file stating honestly: which tools you used and for what, two or three prompts that were decisive, at least two cases where the generated code was wrong or not idiomatic and how you fixed it, and what each member learned. An honest report is graded positively; a report that does not match what happens in the defense is treated as academic dishonesty.

8.2 Individual defense

Each group defends the project in a session of about 15 minutes. The grade is individual: every member is questioned separately, including about parts they did not personally write. Expect questions and live modifications of this kind:

- Add a new type that satisfies `Attender` — for instance a nurse — and make the hospital dispatch it, live.
- Explain what changes if `Person` is embedded by value versus embedded as a pointer.
- Change a method receiver from pointer to value and predict what stops working, and why.
- Explain why a field is lowercase, and what exactly stops the code outside the package from touching it.
- Explain what your program does when two goroutines assign the same room at the same time.
- Justify a design decision: why a map and not a slice, why that return type, why that package split.

8.3 How the defense affects the grade

The group grade obtained with the rubric of Section 9 is multiplied, for each student individually, by the following factor:

Factor	What the student demonstrates during the defense
1.0	Explains any part of the delivered code, including parts they did not write, and completes the live modification correctly.
0.8	Explains their own parts fluently and the rest at a conceptual level; the live modification needs hints.
0.5	Explains only what they personally wrote; cannot justify the design of the rest.
0.0	Cannot explain the delivered code, or the explanation contradicts AI_USAGE.md.

9. Evaluation Rubric

Item	Weight
Entity model — the five structs with their fields and methods, encapsulation respected	**20 %**

Item	Weight
Interfaces and composition — Person embedded, Attender implemented and actually used	**15 %**
The four required queries — correct results and correct return types	**30 %**
Demo program — runs the required scenario, handles the "no room available" case with an error	**15 %**
Idiomatic Go and engineering — package layout, gofmt, go vet, errors, README, comments	**20 %**
Total	**100 %**
Bonus — graphical interface (Fyne, Wails or REST API + HTML/JS frontend)	**+ 10 %**
Bonus — concurrent simulation with goroutines and a mutex, without data races	**+ 5 %**

10. Submission and Final Checklist

- A Git repository with the full history (a single commit on the due date will be asked about in the defense), containing the code, `README.md` and `AI_USAGE.md`.
- The `README.md` states the group members, how to run the program (`go run .`) and how to run the tests (`go test ./...`), plus any design decision worth explaining.

Before submitting, verify that:

- ☐ The project runs from a clean clone with go run . and the tests pass with go test ./...
- ☐ gofmt has been applied and go vet reports nothing.
- ☐ The five structs exist, with unexported fields and methods that access them.
- ☐ Person is embedded in Patient and Doctor, and no field is duplicated.
- ☐ There is a []Attender holding at least two different concrete types.
- ☐ States are typed constants, not loose strings.
- ☐ The four queries return data and are called from main.
- ☐ The "no room available" case returns an error and does not crash the program.
- ☐ README.md and AI_USAGE.md are complete and honest.
- ☐ Every member can explain and modify any part of the project.